

Groupe 5 - iRobot Autonomous Explorer

Master: M1 MOSIG

Team Members: KHOODARUTH Tatiana, GOMEZ ROSALES Laura Sofía, GEBAUER Daniel

Git Repository: <https://github.com/IM2AG-IntroRob2026/Project-Groupe5/tree/main>

Introduction

The project implements an autonomous navigation system for the iRobot Create 3 that goes beyond simple bump-and-turn behavior. It uses a proportional obstacle-avoidance controller and state machine to perceive obstacles at varying distances and scale speed/rotation smoothly, minimizing contact and improving docking reliability. The system also supports timed return-to-dock and an operator override via a keyboard interface.

System specifications

- **Controller:** Dual-node ROS 2 architecture (*explorer* and *keyboard_handler*).
- **Sensors:**
 - 7x IR Intensity Sensors (Readings at ~62Hz).
 - Tactile Bumper Hazards.
 - Docking Status feedback.
- **Visual Feedback:** 6-LED Light Ring (RGB).

System Architecture

The package is organized around one launch entry point and two runtime ROS 2 nodes (*explorer* and *keyboard_handler*) sharing the same namespace. For example, if the namespace is **Robot5**, both nodes use scoped topics such as `/Robot5/mode` and `/Robot5/cmd_vel`, so commands and sensor streams stay bound to that robot instance.

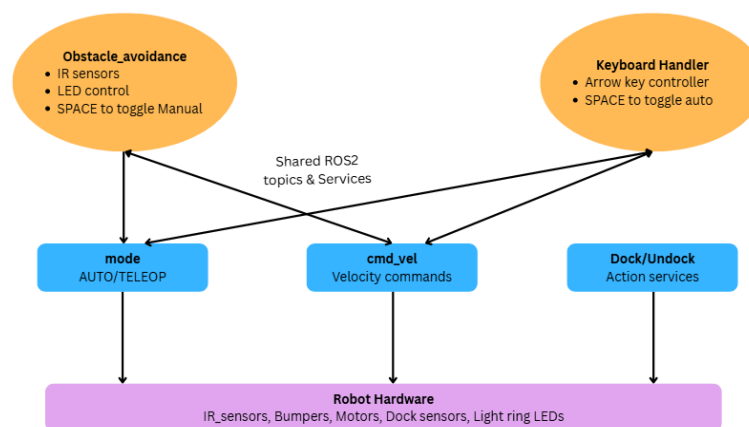


Figure 1: Node Architecture

obstacle_avoidance.py

The system is governed by a State Machine with 7 operational states owning autonomy, avoidance, and action calls.:

1. **DOCKED**: Initial state. Monitor the charging station.
2. **UNDOCKING**: Executes the Undock action sequence.
3. **EXPLORING**: Proportional IR-based navigation.
4. **ESCAPING**: A non-blocking timed window (reverse + turn) triggered by bumper contact.
5. **RETURNING**: Navigates back to the dock while keeping avoidance active.
6. **MANUAL**: Entered by TELEOP, It overrides human control.
7. **COMPLETED**: Mission success state after successful docking.

The **ESCAPING** state uses asynchronous timing (`time.time()`). This allows the robot to continue processing sensor data even while executing recovery maneuvers, ensuring safety at all times.

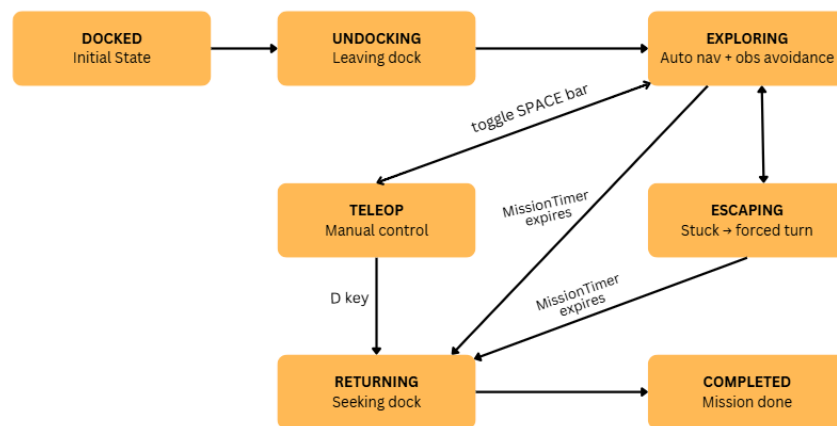


Figure 2: Finite State Machine

keyboard_handler.py

The keyboard node is the operator interface layer:

- Runs non-blocking keyboard capture in a dedicated thread so ROS callbacks continue to run.
- Publishes **mode** commands (TELEOP, AUTO, DOCK) based on key input.
- Publishes manual `cmd_vel` commands from arrow keys only in teleoperation mode.
- Sends a zero-velocity command on mode switches to avoid stale motion carry-over.

explorer.launch.py

The launch file is the runtime orchestration layer. Starts both runtime nodes (explorer and keyboard_handler) with a shared namespace and launch-time parameters:

- Declares shared launch arguments (namespace, linear_speed, angular_speed, exploration_time).
- Starts explorer and keyboard_handler under the same namespace.
- Passes speed and mission parameters into explorer at startup.
- Runs keyboard_handler with TTY emulation and an xterm prefix to support interactive key capture.

Implementation

Proportional Avoidance

We implemented a multi-stage IR perception system using the 7-sensor intensity vector:

- **Aware Zone (IR > 100):** V_EARLY threshold subtle steering adjustment before obstacles are close; speed is only reduced if the side sensors also detect proximity.
- **Early Zone (IR > 150):** EARLY threshold gentle slowdown and turn preparation with yellow LED feedback.
- **Warning Zone (IR > 250):** WARNING threshold proportional braking based on front intensity, with orange LED feedback.
- **Critical Zone (IR > 500):** CLOSE threshold stop forward movement and turn aggressively, with red LED feedback.

We implemented a **Turn-Lock Guard**. If the robot repeatedly hits the red zone while returning to the dock, it briefly forces a pure rotation for 0.6 seconds. This breaks local oscillation and helps the robot re-orient so docking can continue.

Visual Status Ring

The cmd_lightring topic provides formal visual feedback for the robot's operating state. To improve robustness against packet loss, the same color is re-published every 0.3 s:

1. **Green:** Normal navigation or completed mission
2. **Yellow:** Early obstacle awareness
3. **Orange:** Obstacle warning and braking
4. **Red:** Immediate danger or critical obstacle
5. **Purple:** Bumper Recovery active
6. **Blue:** Teleop Mode

Challenges and solutions

Initially, the robot reacted too late to obstacles, because it only began turning once the front IR readings were already close to the critical range. This made wall contact more likely and left too little time for a clean evasive turn.

To address this, we introduced a proportional slowdown strategy using the IR intensity values. As obstacle proximity increases, the robot reduces its linear speed gradually instead of stopping abruptly, which gives the angular controller more time to rotate the robot away from the wall. In the warning zone, the turn command is also strengthened so the robot starts escaping earlier and more decisively.

During return-to-dock operation, we added a turn-lock guard to break repeated red-zone oscillations near corners. When the robot detects the same critical obstacle condition several times in succession, it temporarily switches to pure rotation for 0.6 s so it can re-orient and continue docking.

Testing Methodologies

Mission Reliability

- Multiple runs were performed using the default `exploration_time=60s` parameter. When the robot completes the 60s exploration phase it then begins **RETURNING**. depending on where it is in the map this can add roughly another minute to reach the dock (robot may be far from the base when the timer expires).
- The robot only attempts to dock after the mission timer expires or when a **DOCK/RETURN** command is published via the mode topic. Manual intervention to place the robot near the base is a valid mitigation when recovery is needed.
- **Stuck/corner cases:** if several IR sensors are simultaneously occluded (e.g: chairs closely surrounding the robot), the behavior can oscillate or pause. In tests the pragmatic workaround was to manually free the robot and resume.

Parameter Stress Testing

- **High-speed test:**
 - *linear_speed=0.7, angular_speed=0.65*
 - **Observations:** obstacle detection is less effective at very high forward speeds. Collisions occur more frequently and tend to be harder impacts. After a collision the controller usually recovers by turning $\sim 90^\circ$ and continuing.
- **Low-speed test:**
 - *linear_speed=0.25, angular_speed=0.65*

- **Observations:** lower linear speed reduces bump frequency and gives sensors and avoidance logic more time to react, producing smoother avoidance and docking approaches.

Exploration-duration Tests

We conducted multi-stage testing with variable mission durations (20s, 30s, 60s, and 120s). In all test scenarios, the system demonstrated high temporal reliability, accurately triggering the transition from EXPLORING to RETURNING exactly at the set expiration. Key observations:

- **Return Latency:** The total time required for the **RETURNING** phase varied. If the robot finished its mission far from the Dock then it might take more or less more time depending on how far it is and if there's obstacles in the way of the robot while it's returning.

Docking Scenarios and Recovery

- Docking occurs only after **RETURNING** begins (timer expires) or on explicit **DOCK/RETURN** commands. During docking the avoidance stack remains active. The robot will deviate to avoid obstacles and only send docking goals when the path is reasonably clear.
- The turn-lock guard is effective at reducing repeated red-zone oscillation near corners during **RETURNING**. If the robot becomes physically trapped (sensors contradictory due to partial occlusion), manual (**TELEOP**) was used during tests.

Limitations

- **IR occlusion:** situations where multiple IR sensors are simultaneously blocked (tight corners, tightly grouped furniture) can produce contradictory readings that cause oscillation or temporary stalls.
- **No persistent mapping or path memory:** the controller does not currently use odometry to avoid re-exploring the same local area.
- The dock goal can take an additional ~1 minute to reach the dock as commonly observed depending on robot location.

Possible improvements

1. **Path Memory:** Use odometry / lightweight pose estimation to avoid re-exploring the same 1-meter area repeatedly and to bias RETURNING paths toward known-clear corridors.
2. **Improved Sensor Fusion:** Combine IR, bumpers and short-range lidar or visual cues to reduce ambiguous occlusion cases.

External sources

We inspired ourselves from pre-existing obstacle avoidance implementations.

- https://github.com/enansakib/obstacle-avoidance-turtlebot/blob/master/src/naive_obs_avoid_tb3.py
- https://iroboteducation.github.io/create3_docs/api/docking/
- https://iroboteducation.github.io/create3_docs/api/hazards/